The Benchmark Chooses the Winner: Measuring Fine-Tuning Specialization Across Safety-Guard Benchmarks

Reza Rahimi
reza.rahimi@jazzx.ai
JazzX AI
Los Altos, CA, USA

ABSTRACT

Small language models are increasingly adapted into binary prompt-safety guards, but improvement on benchmark sources represented during training does not establish transferable guard capability. We measure what LoRA-SFT adds beyond the untuned base using paired base-versus-adapted comparisons on four checkpoints from 1.5B to 4B parameters. We hold the prompt interface, training manifest, LoRA capacity, and evaluation rows fixed; repeat training across five seeds; and separate represented-source tests from dataset-held-out benchmark transfer. Evaluation uses macro average precision, per-source effects, calibration-only operating points, and uncertainty over both training runs and evaluation datasets.

Under the locked protocol, SFT changes represented-source macro AP by [RESULT AND INTERVAL] and transfer macro AP by [RESULT AND INTERVAL]. Effects are [stable/heterogeneous] across checkpoints and benchmarks. At a calibration-targeted 5% FPR, [OPERATING-POINT RESULT], with realized test FPR reported rather than assumed.

Prior work has established policy overfitting, calibration sensitivity, and safety degradation after fine-tuning. Our contribution is narrower: a same-checkpoint, same-training-manifest, multi-seed attribution of how compact prompt guards specialize across benchmark sources. The results show that adapting a guard is [FINAL GATED INTERPRETATION].

1 INTRODUCTION

Teams increasingly convert a general instruction model into a safety guard with a short parameter-efficient fine-tune, then report the adapted guard's score on a public benchmark suite. A post-adaptation leaderboard number, however, hides the quantity a practitioner actually needs: what the fine-tune *added* relative to the same checkpoint before adaptation, and whether that addition is competence that transfers to data the guard was not trained on, or specialization toward the sources that happened to be represented during training. Because an adapted guard is usually compared only against *other* models — never against its own untuned base — a gain measured on benchmark sources represented during training is easily mistaken for transferable guard capability.

This paper asks one question: *what does LoRA-SFT add beyond the untuned base model, and does that gain transfer to evaluation datasets whose rows were not used for SFT?* To answer it we run a controlled, same-checkpoint, same-training-manifest, multi-seed study on a fixed four-checkpoint panel (Qwen2.5-1.5B, SmolLM2-1.7B, SmolLM3-3B, Qwen3-4B) and one frozen LoRA-SFT recipe, comparing every adapted guard against its own untuned base across two evaluation regimes: benchmark sources *represented* during training and *dataset-held-out* benchmarks whose rows were withheld from SFT.

Our result-neutral thesis, stated before any scoring, is the following.

> For a fixed four-checkpoint panel and one frozen LoRA-SFT recipe, guard adaptation changes performance differently on represented-source and dataset-held-out tests. Comparing every guard with its own untuned base reveals whether adaptation adds transferable competence or redistributes performance toward sources represented during training.

After scoring, the conclusion follows the preregistered decision rules of Section 6; a heterogeneous or inconclusive outcome is reported as such and is not forced into a negative-transfer narrative.

Scope. The study is deliberately narrow: English input-prompt classification, binary safe/unsafe decision-token scoring, small instruction models from 1.5B to 4B parameters, one fixed LoRA-SFT recipe, three represented-source datasets, four dataset-held-out transfer benchmarks, two single-class stress sets, and a fixed, purposively chosen four-checkpoint panel. It is not a guardrail leaderboard, an objective comparison, an ensembling study, a fairness audit, a frontier-API parity or latency study, or a domain-compliance case study; those questions — including the mortgage-compliance case study — are out of scope here and are treated in separate work. Response, tool-call, and full-dialogue moderation, arbitrary unseen policies, production traffic, and universal claims about fine-tuning are likewise out of scope.

1.1 Contributions

The paper makes exactly three contributions.

- **Controlled base-to-guard attribution.** For each checkpoint, we compare the untuned base with five independently trained SFT adapters on identical examples, prompts, score definitions, and metrics, so any measured change is attributable to the adaptation rather than to a different model, threshold, or evaluation set.
- **Fixed-panel transfer characterization.** We estimate represented-source and dataset-held-out changes separately, report per-base and per-benchmark heterogeneity, and restrict every conclusion to the named four-checkpoint panel and frozen recipe.
- **Auditable measurement package.** We release immutable manifests or redistributable identifiers, source revisions, content hashes, seed-level results, keyed scores,

direct paired intervals, generated tables and figures, and an executable cached-analysis path.

Operating-point evaluation (Section 4.3) is *supporting methodology, not a contribution*: it is a secondary deployment diagnostic and does not provide a distribution-free production guarantee.

1.2 Research questions

The single question above is operationalized as four preregistered research questions.

- **RQ1 (represented-source effect).** For the fixed four-checkpoint panel, how does a common LoRA-SFT recipe change macro average precision on benchmark sources represented during training?
- **RQ2 (benchmark-transfer effect).** How does the same adaptation change macro average precision on dataset-held-out transfer benchmarks?
- **RQ3 (heterogeneity).** Are the represented-source and transfer changes stable across checkpoints and evaluation datasets, or are they concentrated in particular bases and benchmarks?
- **RQ4 (deployment sensitivity).** What TPR/FPR trade-off do the base and SFT systems realize when thresholds are selected on calibration data at a nominal 5% pooled-negative FPR target?

RQ4 is descriptive and secondary: it does not provide a distribution-free production guarantee, and failure to obtain a feasible threshold under the prespecified bound is a reportable outcome rather than grounds for selecting a threshold on test data.

2 RELATED WORK

Small-LLM guard classifiers. A growing family of purpose-built classifiers screens prompts and responses for LLM pipelines — Llama Guard [10, 15], ShieldGemma [18], Granite Guardian [16], and WildGuard [7] — typically framing moderation as supervised classification that emits a verdict token or score. Efficiency-oriented work argues that small, cheap classifiers can serve as practical guards [14], and GuardBench shows that a general instruction-tuned model can rival guard-specific fine-tunes on a large suite [3]. Our guard formulation is deliberately minimal — a single-token {safe, unsafe} logprob head read in one forward pass, LoRA-tuned from a compact instruction base [2, 9] — because the object of study is the *before-versus-after* effect of adaptation, not a new architecture.

Fine-tuning degradation and benchmark/policy transfer. Prior work establishes that fine-tuning can degrade or collapse a model's safety behavior, and analyzes when this happens through alignment/fine-tuning similarity [8]. Policy-overfitting and unseen-policy adaptation are studied directly by augmented policy training for domain-generalizable guardrails [12], and specialized-domain moderation is the subject of expert-annotated benchmarks such as ExpGuard [5]. Detector rankings have been shown to be regime-dependent, with no single model dominating across deployment settings [1], and fine-tuning objectives themselves shape safety and robustness [17]. We credit these results as prior art: policy overfitting, transfer instability, and objective- and data-dependent degradation are established. Our study does not re-derive them; it isolates and quantifies one narrow slice — the paired base-to-SFT change on a fixed panel, reported separately for represented and dataset-held-out sources.

Calibration and operating points. Guard scores require calibration to be read as probabilities or thresholds; temperature scaling [6] and guard-specific calibration analyses [13] are standard, as is continuous risk scoring with strictness-adaptive thresholds [4]. Calibrated ensembles can mitigate accuracy trade-offs under distribution shift [11]. We use calibration only as a secondary operating-point diagnostic (Section 4.3) fit on held-back calibration rows, never on transfer or stress data, and we do not propose a new calibration method.

Novelty boundary. Unlike work proposing new policy augmentation, calibration, or preservation objectives, we isolate the before-versus-after effect of a common SFT guard recipe on the same compact checkpoints and report represented-source and dataset-held-out changes separately. We do not claim a new metric, calibration algorithm, preservation loss, preference-learning method, ensemble, compliance architecture, or theorem. The defensible novelty is a controlled, same-checkpoint, same-training-manifest, multi-seed estimate of how compact binary prompt guards move between represented-source discrimination and dataset-held-out benchmark transfer.

3 CONTROLLED STUDY DESIGN

Figure 1 summarizes the pipeline: pinned source snapshots are compiled into frozen manifests, audited for overlap and license, and locked; four bases are each adapted with five SFT seeds; the base and adapted guards are scored on the represented-source, transfer, and stress manifests; and paired deltas feed the preregistered claim gates. No stage reads evaluation labels before the final scoring pass.

3.1 Prompt-safety task and label mapping

The guard is a binary classifier that emits a single-token verdict, `safe` or `unsafe`. A prompt is `unsafe` if it should be blocked (harmful content, successful jailbreak/injection, policy-violating request) and `safe` otherwise. Each source's native label is mapped to this binary target by a frozen rule: ToxicChat `toxicity`= 1 →unsafe; `deepset/prompt-injections` `label`= 1 →unsafe; `jailbreak-classification` `type` beginning *jailbreak*→unsafe; JailbreakBench harmful/benign splits; XSTest contrast prompts→unsafe; WildGuardTest `prompt_harm_label`; WildJailbreak integer label; OR-Bench-Hard benign portion (safe); HarmBench (unsafe). All native-to-binary maps, text fields, and per-source deterministic targets are fixed in `configs/paper_a_sft.yaml` and recorded in the manifest.

3.2 Guard formulation: a single-token logprob head

The guard treats safety classification as a single next-token prediction. A prompt x is wrapped in one versioned instruction template shared across every base and seed, and the model is asked for a one-word verdict. Rather than sampling text, we read the last-position logits and restrict attention to the two verdict tokens. Let $z_{\text{safe}}(x, t)$

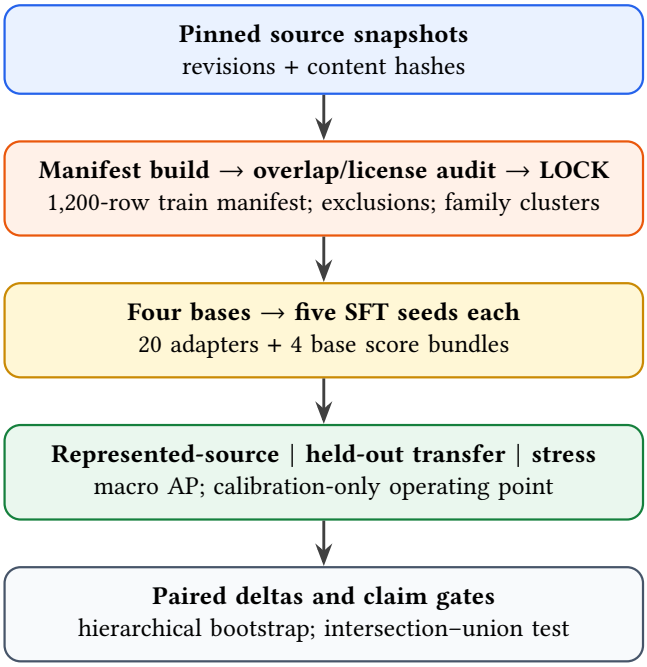


Figure 1: Study design. Source snapshots are frozen and audited before any training; evaluation manifests are scored only after all twenty adapters and four base bundles exist. Paired base-versus-SFT deltas drive the preregistered claim gates (Section 6); this figure carries no result numbers.

and $z_{\text{unsafe}}(x,t)$ be those logits at the final prompt position t. The stored canonical score is the logit difference

$$s(x) = z_{\text{unsafe}}(x,t) - z_{\text{safe}}(x,t), \tag{1}$$

and the derived probability is the two-way softmax

$$p(\text{unsafe} \mid x) = \frac{\exp(z_{\text{unsafe}})}{\exp(z_{\text{safe}}) + \exp(z_{\text{unsafe}})}. \tag{2}$$

Raw `safe`/`unsafe` logits are the canonical stored values; probabilities and any temperature-scaled calibration are derived from them. Generated verdict text is never used as the primary score. For each checkpoint we verify that the decision tokens are distinct single tokens under a fixed convention (leading-space `" safe"`/`" unsafe"` first, no-leading-space fallback), and we store token IDs, token strings, truncation state, and scored/original token counts. Training, scoring, and tests import the same prompt/token specification.

3.3 Training manifest and exclusions

Training reads one frozen 1,200-row manifest and never resamples live datasets. The manifest draws **400 rows each** from three represented sources — ToxicChat, Prompt-Injections, and Jailbreak-Classification — with **200 safe and 200 unsafe** rows per source, selected deterministically by a frozen SHA-256 hash-ranking rule salted with `data_seed` and shared as one fixed row order across every checkpoint and training seed. At effective batch size 4, 1,200 rows produce exactly 300 optimizer updates for one complete exposure of the manifest.

BeaverTails and OR-Bench are excluded from training. BeaverTails' `is_safe` annotation labels the prompt–response interaction rather than the prompt alone, and OR-Bench is reserved for the benign stress set and shares an evaluation family; including either would confound the prompt-only, dataset-held-out design. The study uses the non-commercial academic data branch (ToxicChat is CC BY-NC 4.0), so any released adapter is marked non-commercial and third-party raw text is redistributed as pinned identifiers, revisions, and content hashes rather than under the repository license.

Decontamination audit. Before locking, an audit compiles union family clusters across all candidate rows: an edge is added between rows sharing an authoritative upstream family/conversation/pair/scenario ID and between rows whose character 5-gram MinHash (256 permutations) estimates Jaccard similarity $\geq$ 0.85; connected components define `family_id`; every component crossing train and any evaluation split is adjudicated, and the train-side member of an unresolved cross-split component is removed. The audit enforces hard assertions — `or_bench_train_count=` 0, `beavertails_train_count=` 0, exact train/eval overlap = 0, conflicting-label overlap = 0, and complete revision/hash/near-duplicate-disposition coverage — and reports sensitivity counts at similarity 0.80 and 0.90. On the current build the audit reports **0 exact train/eval overlap**, **0 conflicting-label overlap**, and the removal of **4 WildJailbreak conflict rows** from the training candidates. Because these are audited data-pipeline properties rather than model outcomes, they are stated here; all post-audit split counts are published in the manifest report.

3.4 Model panel and frozen SFT recipe

The estimand is a fixed, purposively chosen four-checkpoint panel; we do not bootstrap model identities or infer over model families. All four checkpoints, their pinned revisions, and the locked LoRA-SFT recipe are in Table 2. Each base is adapted with five training seeds (42–46) sharing a fixed data-order seed (42), giving 20 adapters; the four untuned bases are scored once each and reused across comparisons (4 bases +20 adapters = 24 model bundles). The recipe is frozen before final scoring: completion-only SFT with the loss on the verdict token plus the appended EOS token, LoRA r=32, α=64, dropout 0.05 on the q/k/v/o/gate/up/down projections, 300 optimizer updates, learning rate 2×10^{-4} with cosine schedule and 0.03 warmup, per-device batch 1 with gradient accumulation 4 (effective batch 4), and maximum sequence length 1,024.

3.5 Evaluation regimes

We use precise regime terminology and avoid "fair," "uncontaminated," and "source-family OOD." Three regimes partition evaluation.

- **Represented-source** = {ToxicChat, Prompt-Injections, Jailbreak-Classification} test rows, held back from training. These measure discrimination on sources represented during training.
- **Dataset-held-out transfer** = {JailbreakBench, XSTest, WildGuardTest, WildJailbreak}, whose rows were withheld from SFT. These benchmarks use heterogeneous

Table 1: Data sources and manifest roles. Train counts are locked design values (400 per source; 200 safe / 200 unsafe). Transfer and stress counts are the deterministic per-label targets from `configs/paper_a_sft.yaml`; represented-source test rows are split 40/60 into calibration and represented-source ID, and their final post-audit counts are published in the manifest report. Model-checkpoint revisions are in Table 2; dataset revisions and content hashes are pinned in the config. "see manifest" marks a license that is recorded and snapshotted at lock time.

Source (Hugging Face)	Native label origin	Role	Count (target)	License
`lmsys/toxic-chat`	prompt toxicity	train + calibration/ID	400 train (200:200)	CC BY-NC 4.0
`deepset/prompt-injections`	prompt injection	train + calibration/ID	400 train (200:200)	see manifest
`jackhhao/jailbreak-classification`	prompt jailbreak	train + calibration/ID	400 train (200:200)	see manifest
`JailbreakBench/JBB-Behaviors`	harmful/benign split	transfer (test only)	120 (60:60)	see manifest
`natolambert/xstest-v2-copy`	contrast prompt	transfer (test only)	240 (120:120)	see manifest
`allenai/wildguardmix`	prompt harm label	transfer (test only)	800 (400:400)	ODC-BY + AI2
`allenai/wildjailbreak`	adversarial label	transfer (test only)	420 (≈210:210)	ODC-BY + AI2
`bench-llm/or-bench` (hard-1k)	benign over-refusal	stress: benign FPR only	400 (0:400)	see manifest
`walledai/HarmBench`	harmful	stress: recall only	200 (200:0)	see manifest

Table 2: Fixed model panel and frozen LoRA-SFT recipe. Revisions are pinned; the run fails rather than falling back to a moving `main`. Decision tokens are verified distinct single tokens per checkpoint under the leading-space-first convention (exact token IDs recorded in `LOCK.json`). Trainable-parameter counts, training tokens, and wall time are per-run metadata filled from run logs after training; they are not evaluation results.

Checkpoint	Params	Revision (pinned)	Decision tokens	Trainable / tokens / time
Qwen/Qwen2.5-1.5B-Instruct	1.5B	`989aa79…71aa306`	{safe, unsafe} single-token	`[RESULT]` / `[RESULT]` / `[RESULT]`
HuggingFaceTB/SmolLM2-1.7B-Instruct	1.7B	`31b70e2…46d38674`	{safe, unsafe} single-token	`[RESULT]` / `[RESULT]` / `[RESULT]`
HuggingFaceTB/SmolLM3-3B	3B	`a07cc9a…335b0ac1`	{safe, unsafe} single-token	`[RESULT]` / `[RESULT]` / `[RESULT]`
Qwen/Qwen3-4B	4B	`1cfa9a7…03b3df60c`	{safe, unsafe} single-token	`[RESULT]` / `[RESULT]` / `[RESULT]`

Recipe (all cells): completion-only SFT (verdict + EOS); LoRA r=32, α=64, dropout 0.05; q,k,v,o,gate,up,down; 300 steps; lr 2×10^{-4} cosine, warmup 0.03; effective batch 4; max len 1,024; seeds 42–46; data-order seed 42.

native label constructs and may share conceptual or data-generation lineage, so we study *dataset-held-out benchmark transfer*, not source-family independence or a single fixed policy.

- **Stress** = {OR-Bench-Hard benign portion (false-positive rate only), HarmBench (recall only)}. These are single-class sets: **no AP or AUROC is computed on stress data**, and the confounded OR-Bench-Hard benign/unsafe hybrid is never used for AP.

The primary metric is benchmark-macro average precision within each regime; the secondary metric is the calibration-targeted 5% FPR operating point (Section 4.3).

3.6 Estimands and statistical protocol

For regime R, checkpoint b, and training seed r, the per-regime metric is the macro average precision over that regime's benchmarks,

$$M_R(b, r) = \frac{1}{|K_R|} \sum_{k \in K_R} \mathrm{AP}_k(b, r), \tag{3}$$

computed with the tie-aware, non-interpolated `scikit-learn` definition. The base-to-SFT change for a cell is

$$\Delta_R(b, r) = M_R(\mathrm{SFT}_{b,r}) - M_R(\mathrm{base}_b), \tag{4}$$

and the fixed-panel aggregate averages the four checkpoint deltas without resampling checkpoint identities,

$$\bar{\Delta}_R = \frac{1}{4} \sum_b \left[\tfrac{1}{5} \textstyle\sum_r M_R(\mathrm{SFT}_{b,r}) - M_R(\mathrm{base}_b) \right]. \tag{5}$$

The primary result is the joint vector $\theta = (\bar{\Delta}_{\text{represented}}, \bar{\Delta}_{\text{transfer}})$; we do not collapse it into a single scalar "specialization score" except as an optional visualization.

Uncertainty. We separate training-seed uncertainty from conditional evaluation-set uncertainty with a hierarchical paired bootstrap: 10,000 replicates (RNG seed 20260712), the four checkpoint identities fixed in every replicate, five SFT seed indices resampled with replacement within each checkpoint, and one Poisson(1) weight drawn per global `family_id` and applied to all of that family's rows across datasets (preserving cross-label, cross-dataset, and paired base/SFT dependence). We report all five seed values, per-base and fixed-panel intervals, leave-one-transfer-benchmark-out and leave-one-base-out sensitivities (descriptive, not population inference), and per-base/per-benchmark heterogeneity tables.

Claim family and mode. The primary family is the pair (represented-source macro-AP delta, transfer macro-AP delta), and the specialization claim is an intersection–union test in which both one-sided component nulls must be rejected at α=0.05. `LOCK.json` records `analysis_mode` as `powered_confirmatory` or `precision_focused`; in the latter, the same intervals are computed but reported with estimation language rather than

formal rejection claims. We do not use McNemar as a test of AP, infer significance from non-overlapping marginal intervals, treat seeds as model families, or select any benchmark, seed, base, or threshold after viewing final results.

4 RESULTS

This section is a preregistered skeleton. Every numeric cell marked `[RESULT]` *or bracketed placeholder is filled from* `artifacts/paper_a_sft/analysis` *after the locked run; no value below is a final measurement, and the earlier pilot figures (e.g. the 0.884 vs. 0.780 base/SFT aggregate) are explicitly not reused as final evidence.*

4.1 Primary fixed-panel result (RQ1, RQ2)

Table 3 reports, per base, the untuned-base and mean-SFT macro AP in each regime, the paired base-to-SFT delta with its one-sided 95% interval, and the fixed-panel aggregate $\theta = (\bar{\Delta}_{\text{represented}}, \bar{\Delta}_{\text{transfer}})$. Per-seed values for every cell are in the appendix seed table (Section 6 references the gate decision).

4.2 Per-base and per-benchmark heterogeneity (RQ3)

The aggregate deltas are decomposed into per-benchmark paired deltas and cross-cell heterogeneity (range and standard deviation across bases and across benchmarks, the base-by-benchmark sign table, and every leave-one-out aggregate). Table 4 lists the per-benchmark deltas; the aggregate sign is labeled *sensitivity-stable* only when every leave-one-out estimate shares the sign of the full estimate. RQ3 remains descriptive, and no post-hoc subgroup is promoted to a confirmatory endpoint.

4.3 Secondary operating point at a calibration-targeted 5% FPR (RQ4)

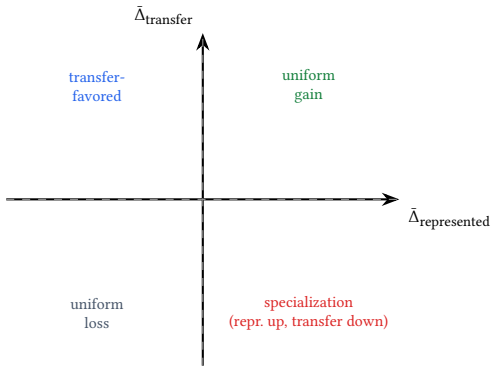
As a deployment diagnostic only, we fit one positive temperature per base and per adapter on calibration rows (never on transfer or stress rows), then select the threshold that maximizes calibration recall subject to a one-sided 95% Clopper–Pearson upper bound of at most 5% on pooled calibration-negative FPR; if none is feasible the cell reports `NO_FEASIBLE_THRESHOLD`. We report the realized represented-source and transfer FPR (with intervals) rather than describing test FPR as matched, and we report the paired ΔTPR between SFT and base per regime. The lower panel of Table 4 carries these values.

4.4 Stress-set results

The two single-class stress sets are reported as one-sided diagnostics only: the OR-Bench-Hard benign false-positive rate (over-refusal stress) and HarmBench recall at the frozen threshold (positive stress). Neither contributes to any AP aggregate. These appear in the lower panel of Table 4.

4.5 Specialization plane

Figure 2 shows the interpretation plane used to read the joint result: the horizontal axis is the represented-source macro-AP delta, the vertical axis is the transfer macro-AP delta, one point per seed



(per-seed points and panel mean plotted from analysis artifacts)

Figure 2: Specialization plane (schematic). Each seed of each checkpoint is one point; the four quadrants give the result-neutral interpretation. Points and the fixed-panel mean are generated from `artifacts/paper_a_sft/analysis/figures` after the locked run.

is colored by checkpoint, and the zero lines create four quadrants (uniform gain, specialization, uniform loss, and transfer-favored). The fixed-panel mean is drawn separately. The point cloud is populated from analysis artifacts after the locked run; the axes and quadrant interpretation are result-independent.

5 LIMITATIONS

We frame this as a controlled measurement study, and several caveats bound its interpretation.

Heterogeneous native policies. The transfer benchmarks encode different native label constructs (harm, refusal, jailbreak, contrast), so their macro-average mixes distinct policies rather than measuring one fixed policy.

Fixed two-lineage panel. The panel is four purposively chosen checkpoints spanning two model lineages (Qwen and SmolLM); the estimand is this panel, and results do not extrapolate to other architectures, scales, or vendors.

Previously inspected benchmarks. The evaluation datasets are public and have been examined during pipeline development, so they are held out from *training* but are not novel to the field; measured transfer is dataset-held-out, not a guarantee against distributional familiarity.

Balanced prevalence. Represented-source and transfer test pools are near-balanced, whereas inline screening usually runs at much lower unsafe prevalence; precision, F1, and AP are prevalence-dependent, so balanced-set numbers overstate production precision at a fixed FPR.

Non-commercial and gated data. The study uses the non-commercial academic branch (ToxicChat is CC BY-NC 4.0) and gated AI2 sources (WildGuardMix, WildJailbreak), so redistribution is via identifiers, revisions, and hashes rather than raw text, and released adapters are non-commercial.

Prompt-only scope. The guard classifies input prompts only; response, tool-call, and full-dialogue moderation are out of scope and would require re-running the same discipline on those surfaces.

Table 3: Primary fixed-panel result. Base and mean-SFT benchmark-macro AP, with paired base-to-SFT deltas and one-sided 95% intervals, for the represented-source and dataset-held-out transfer regimes. The last row is the fixed-panel aggregate θ. All values are filled from `artifacts/paper_a_sft/analysis` after the locked run; full five-seed values per cell are in the appendix.

	Represented-source macro AP			Transfer macro AP		
Checkpoint	base	SFT (mean)	Δ [95% CI]	base	SFT (mean)	Δ [95% CI]
Qwen2.5-1.5B	[RESULT]	[RESULT]	[RESULT] [CI]	[RESULT]	[RESULT]	[RESULT] [CI]
SmolLM2-1.7B	[RESULT]	[RESULT]	[RESULT] [CI]	[RESULT]	[RESULT]	[RESULT] [CI]
SmolLM3-3B	[RESULT]	[RESULT]	[RESULT] [CI]	[RESULT]	[RESULT]	[RESULT] [CI]
Qwen3-4B	[RESULT]	[RESULT]	[RESULT] [CI]	[RESULT]	[RESULT]	[RESULT] [CI]
Fixed-panel aggregate $\bar{\Delta}_R$	—	—	**[RESULT] [CI]**	—	—	**[RESULT] [CI]**

Table 4: Per-benchmark and operating-point sensitivity. Upper panel: per-source paired base-to-SFT AP delta for each represented and transfer benchmark. Lower panel: calibration-targeted 5% FPR operating point (TPR and realized FPR, base vs. SFT) and single-class stress diagnostics. All values are filled from `artifacts/paper_a_sft/analysis` after the locked run.

Metric	Regime	base	SFT	Δ [95% CI]
ToxicChat AP	repr.	[RESULT]	[RESULT]	[RESULT] [CI]
Prompt-Injections AP	repr.	[RESULT]	[RESULT]	[RESULT] [CI]
Jailbreak-Class. AP	repr.	[RESULT]	[RESULT]	[RESULT] [CI]
JailbreakBench AP	transfer	[RESULT]	[RESULT]	[RESULT] [CI]
XSTest AP	transfer	[RESULT]	[RESULT]	[RESULT] [CI]
WildGuardTest AP	transfer	[RESULT]	[RESULT]	[RESULT] [CI]
WildJailbreak AP	transfer	[RESULT]	[RESULT]	[RESULT] [CI]
Represented TPR @cal-5% FPR	repr.	[RESULT]	[RESULT]	[RESULT] [CI]
Transfer TPR @cal-5% FPR	transfer	[RESULT]	[RESULT]	[RESULT] [CI]
Realized FPR (represented)	repr.	[RESULT]	[RESULT]	[RESULT]
Realized FPR (transfer)	transfer	[RESULT]	[RESULT]	[RESULT]
OR-Bench-Hard benign FPR	stress	[RESULT]	[RESULT]	[RESULT]
HarmBench recall	stress	[RESULT]	[RESULT]	[RESULT]

Finite seeds and datasets. Five training seeds per checkpoint and a small number of evaluation datasets bound statistical precision; the analysis reports seed-level values and leave-one-out sensitivities so readers can judge stability rather than treating the aggregate as a population estimate.

6 CONCLUSION

This paper measures one thing carefully: what a frozen LoRA-SFT recipe adds beyond the untuned base on a fixed four-checkpoint panel, reported separately for represented-source and dataset-held-out transfer. The interpretation follows the preregistered decision tree rather than a preferred narrative, and the mode recorded in `LOCK.json` governs whether the language is confirmatory or estimation-only.

- **Gate A (represented-source).** If the one-sided 95% lower bound of $\bar{\Delta}_{\text{represented}}$ exceeds 0, we state that, for this fixed panel and recipe, SFT improved represented-source macro AP; otherwise we state that the study did not establish a panel-wide represented-source improvement.
- **Gate B (transfer).** If the one-sided 95% upper bound of $\bar{\Delta}_{\text{transfer}}$ is below 0 *and* no leave-one-transfer-benchmark-out or leave-one-base-out aggregate reverses the sign, we state that SFT reduced dataset-held-out transfer macro AP; otherwise we report the measured heterogeneity.
- **Specialization trade-off.** Only if Gates A and B both pass do we state that the fixed panel exhibits an in-source/transfer specialization trade-off under the frozen recipe. We do not claim that fine-tuning universally hurts transfer, that SFT is intrinsically non-robust, that all guard models specialize, or that a causal mechanism is proven.

The result-neutral takeaways stand regardless of which branch obtains: adaptation is not a guaranteed uniform upgrade, and a guard should always be evaluated against its own untuned base across both represented-source and dataset-held-out transfer regimes before its benchmark number is read as capability. The final gated statement is that adapting a guard is [FINAL GATED INTERPRETATION].

A PROVENANCE AND STATISTICAL APPENDIX

The released artifact tree under `artifacts/paper_a_sft/` contains the material that regenerates every result: the frozen manifests (`train`, `calibration`, `id_test`, `transfer_test`, and the two stress manifests) with the canonical per-row schema (sample ID, source, revision, split, label and provenance, content hash, family ID, license, redistribution class, and overlap disposition); the decontamination audit (`audit.json`/`audit.md`) with exact-overlap, conflicting-label, near-duplicate, family-cluster, and license inventories plus 0.80/0.90 sensitivity counts; `LOCK.json`

recording git SHA, model/tokenizer/data revisions, manifest and prompt hashes, the recipe, seeds, metrics, target FPR, primary contrasts, `analysis_mode`, and the power/precision report hash; per-run metadata and adapter hashes for all twenty cells and four base bundles; the keyed per-row score table (Parquet plus immutable metadata) with the full score schema; and the analysis outputs (`results.json`, `claim_checks.json`, `seed_values.csv`, `per_benchmark.csv`, `sensitivity.json`, and generated tables and figures). Full five-seed values per cell, full per-source tables, calibration diagnostics (NLL and Brier before/after temperature scaling, per-source and macro FPR with a family-weighted upper bound), model/tokenizer/prompt hashes, and implementation/hardware details are placed here rather than in the main text. All main-text tables and figures are generated from these artifacts; `[RESULT]` placeholders are filled by the cached-analysis path after the locked run.

B OPTIONAL EXTERNAL VALIDATION: EXPGUARD

As an optional external check on any specialization finding, the expert-annotated specialized-domain moderation benchmark ExpGuard [5] provides an independent, differently sourced prompt-moderation set spanning specialized domains. Re-running the same paired base-versus-SFT comparison on ExpGuard — scoring the input prompt only, with the identical single-token log-prob head — would test whether the panel's represented/transfer pattern reproduces on an external expert-labeled source. We report this as external validation rather than a primary result: the outcome is filled from the cached-analysis path (base vs. SFT paired AP delta with interval) after the locked run and is not part of the preregistered claim family.

REFERENCES

[1] Akindoyin Akinrele and Shreyank N. Gowda. 2026. Prompt Injection Detection is Regime-Dependent: A Deployment-Aware Evaluation with Interpretable Structural Signals. arXiv:2605.26999. arXiv:2605.26999 [cs.CL]

[2] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlíček, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2025. SmolLM3: smol, multilingual, long-context reasoner. Hugging Face blog + model card (HuggingFaceTB/SmolLM3-3B), https://huggingface.co/blog/smollm3.

[3] Elias Bassani and Ignacio Sanchez. 2024. GuardBench: A Large-Scale Benchmark for Guardrail Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 18393–18409.

[4] Ding, Li, Lu, and Shi. 2026. FlexGuard: Continuous Risk Scoring for Strictness-Adaptive LLM Content Moderation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*. arXiv:2602.23636 [cs.CL] ACL 2026; arXiv:2602.23636.

[5] ExpGuard 2026. ExpGuard: LLM Content Moderation in Specialized Domains. arXiv:2603.02588 [cs.CL] ICLR 2026; arXiv:2603.02588.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *ICML 2017 (PMLR 70)*. 1321–1330. arXiv:1706.04599

[7] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. 2024. WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs. In *NeurIPS 2024 (Datasets & Benchmarks Track)*. arXiv:2406.18495

[8] Lei Hsiung, Tianyu Pang, Yung-Chen Tang, Linyue Song, Tsung-Yi Ho, Pin-Yu Chen, and Yaoqing Yang. 2026. Why LLM Safety Guardrails Collapse After Fine-tuning: A Similarity Analysis Between Alignment and Fine-tuning Datasets. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 16601–16619. https://doi.org/10.18653/v1/2026.acl-long.756 arXiv:2506.05346 Earlier workshop version at ICML 2025 DIG-BUGS.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR 2022*. arXiv:2106.09685

[10] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint* (2023). arXiv:2312.06674

[11] Ananya Kumar, Tengyu Ma, Percy Liang, and Aditi Raghunathan. 2022. Calibrated ensembles can mitigate accuracy tradeoffs under distribution shift. In *Uncertainty in Artificial Intelligence (UAI), PMLR 180*. 1041–1051.

[12] Liu, Baldini, Rabinowitz, Rosenberg, Gehrmann, and Dredze. 2026. Domain Generalizable AI Guardrails with Augmented Policy Training. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*. ACL Anthology 2026.acl-long.748.

[13] Hongfu Liu, Hengguan Huang, Xiangming Gu, Hao Wang, and Ye Wang. 2025. On Calibration of LLM-based Guard Models for Reliable Content Moderation. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.10414.

[14] Vasudev Majhi, Dhruv Gupta, Advait Singh, Matthew Barker, and Dhruv Kumar. 2025. Do You Really Need a GPU to Guard Your LLM? CPU-Class Classifiers and Multi-Stage Pipelines for Safety Enforcement at Scale. *arXiv preprint* (2025). arXiv:2512.19011

[15] Meta AI. 2024. Llama Guard 3-1B Model Card. Hugging Face model card. https://huggingface.co/meta-llama/Llama-Guard-3-1B.

[16] Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehling, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Zahra Ashktorab, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, Ambrish Rawat, Kush R. Varshney, and Prasanna Sattigeri. 2025. Granite Guardian: Comprehensive LLM Safeguarding. In *NAACL 2025 (Industry Track)*. 607–615. arXiv:2412.07724

[17] Daniel Vennemeyer, Punya Syon Pandey, Phan Anh Duong, Michael Umeokoli, and Samuel Ratnam. 2026. Objective Matters: Fine-Tuning Objectives Shape Safety, Robustness, and Persona Drift. arXiv:2601.12639. arXiv:2601.12639 [cs.CL]

[18] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. 2024. ShieldGemma: Generative AI Content Moderation Based on Gemma. *arXiv preprint* (2024). arXiv:2407.21772